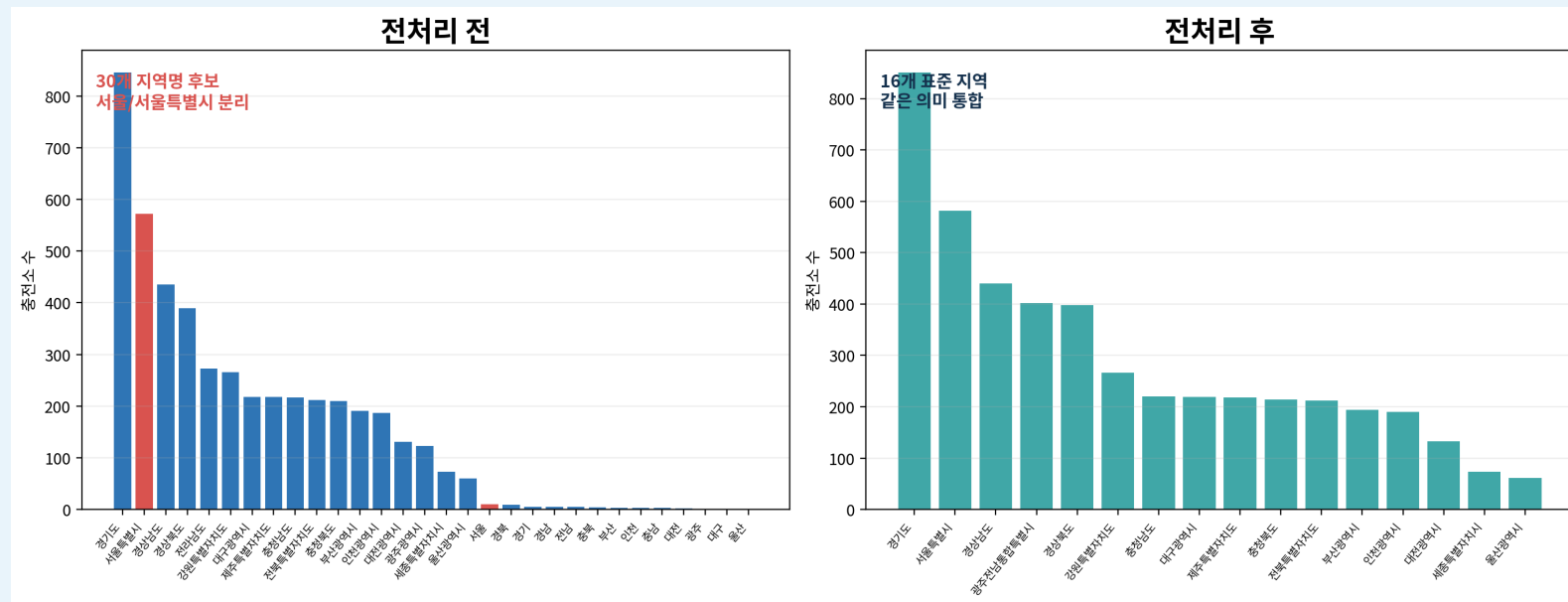


전처리 전과 후, 그래프가 어떻게 달라질까?

전기차 충전소 데이터로 배우는
지역명 전처리,
막대그래프,
지도 시각화



수업의 핵심 흐름

02

학생들이 전처리의 필요성을 그래프에서 직접 발견하게 합니다.

1. 원본 데이터 확인
2. 전처리 전 막대그래프 그리기
3. 서울/서울특별시가 분리되는 문제 발견
4. 결측치·중복·지역명 표준화 수업
5. 전처리 후 막대그래프와 비교
6. 위도·경도를 활용한 지도 시각화

데이터를 그냥 믿지 않는다

→ 그래프로 확인한다

→ 문제를 발견한다

→ 전처리한다

→ 다시 시각화한다

STEP 1. 데이터 불러오기

먼저 데이터의 크기와 앞부분을 확인합니다.

```
import pandas as pd
df = pd.read_csv(
    "한국전력공사_전기차충전소위경도_시도.csv",
    encoding="cp949"
)

print("데이터의 크기:", df.shape)
print(df.head())
```

데이터 크기: 4,673행 × 5열

주요 열: 충전소명, 시도, 위도, 경도

시도 열에는 지역명뿐 아니라

주소가 함께 들어 있음

분석 전에 먼저
“이 열이 정확히 무엇을 담고 있는가?”
를 확인합니다.

STEP 2. 전처리 전 데이터 확인

주소 전체를 그래프로 그리면 너무 많은 막대가 생기므로, 첫 단어만 지역명 후보로 추출합니다.

```
지역별_개수_전 = df["시도"].str.split().str[0]  
  
print(지역별_개수_전)
```

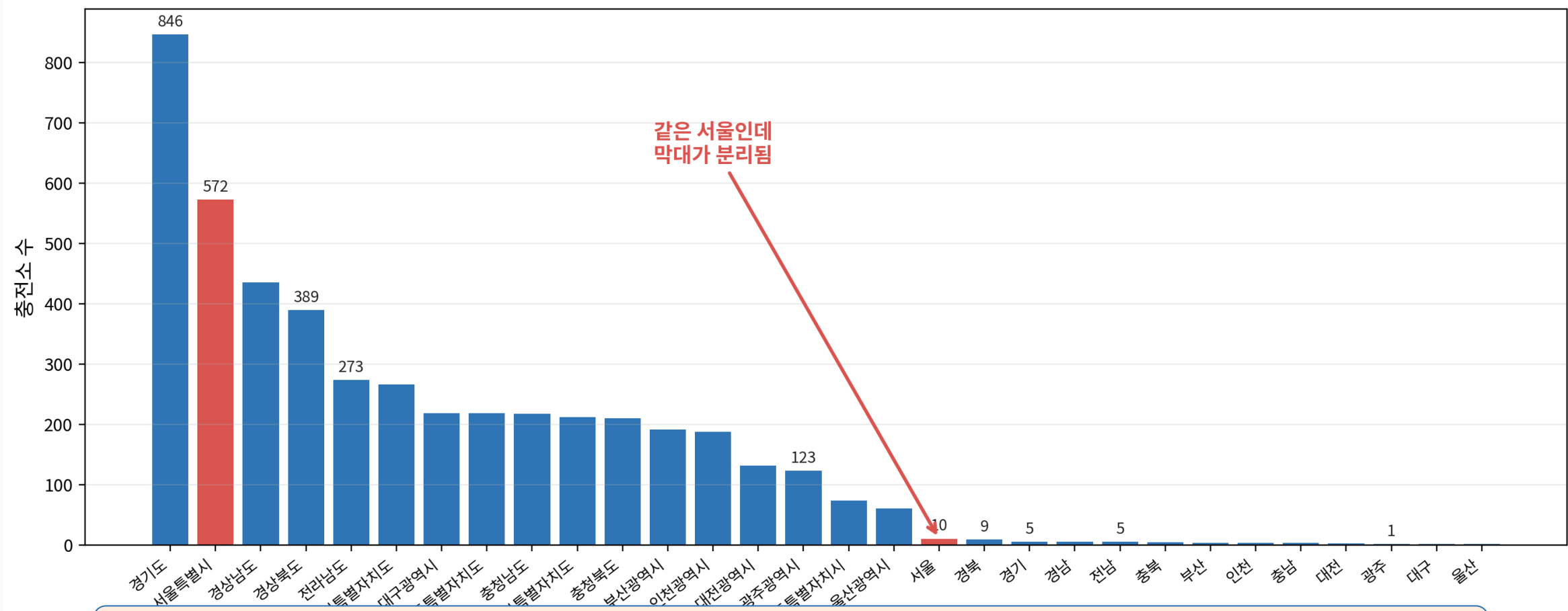
아직 표준화하지 않은 상태
서울특별시와 서울이
-----서로 다른 값으로 남아 있음

이 단계는 “완성된 전처리”가 아니라 “문제를 보여 주기 위한 전처리 전 시각화”입니다.

STEP 3. 전처리 전 막대그래프

표기가 섞이면 같은 지역도 서로 다른 막대로 나타납니다.

전처리 전: 표기가 섞인 지역별 충전소 수



서울특별시와 서울은 같은 지역인데 왜 그래프에서는 따로 보일까?

문제 발견: 컴퓨터는 의미를 이해하지 못한다

06

사람은 같은 지역으로 이해하지만, 컴퓨터는 문자열이 다르면 다른 값으로 계산합니다.

원본 표기	컴퓨터의 판단	문제
서울특별시	서울특별시	표준 표기
서울	서울	서울특별시와 별개 값
경상북도	경상북도	표준 표기
경북	경북	경상북도와 별개 값

그래서 지역명을 하나의 기준으로 맞추는 “표준화”가 필요합니다.

STEP 4. 결측치와 중복 확인

지역명을 고치기 전에 기본적인 데이터 품질을 먼저 점검합니다.

```
# 결측치 확인
print(df[" 시도"].isna().sum())
print(df[df[" 시도"].isna()])

# 중복 확인
print(df.duplicated().any())
print(df.duplicated().sum())
print(df[df.duplicated()])

# 필요할 때만 처리
# df = df.dropna(subset=[" 시도"])
# df = df.drop_duplicates()
```

결측치: 비어 있는 값

중복: 같은 행이 반복된 값

처리 전 반드시 원본 행을 확인

무조건 삭제하지 않고 분석 목적에 맞게 판단

STEP 5. 주소에서 시도명 추출 + 표준화

같은 의미의 지역명을 하나의 표준값으로 연결합니다.

```
# 열 이름 정리 + 주소에서 시도명 뽑기
df = df.rename(columns={"시도": "주소"})
df["시도명"] = df["주소"].str.split().str[0]

시도_표준화 = {
    "서울": "서울특별시", "부산": "부산광역시",
    "대구": "대구광역시", "인천": "인천광역시",
    "대전": "대전광역시", "울산": "울산광역시",
    "세종": "세종특별자치시", "경기": "경기도",
    "강원": "강원특별자치도", "강원도": "강원특별자치도",
    "충북": "충청북도", "충남": "충청남도",
    "전북": "전북특별자치도", "전라북도": "전북특별자치도",
    "전남": "전남광주특별시", "광주": "전남광주특별시",
    "광주광역시": "전남광주특별시", "전라남도": "전남광주특별시",
    "경북": "경상북도", "경남": "경상남도",
    "제주": "제주특별자치도", "제주도": "제주특별자치도"
}

df["시도명"] = df["시도명"].replace(시도_표준화)
```

```
df["시도명"] = df["시도명"].replace(시도_표준화)
```


STEP 6. 전처리 후 막대그래프

09

이번에는 표준화된 시도명 열로 다시 집계합니다.

```
지역별_개수 = df["시도명"].value_counts()
```

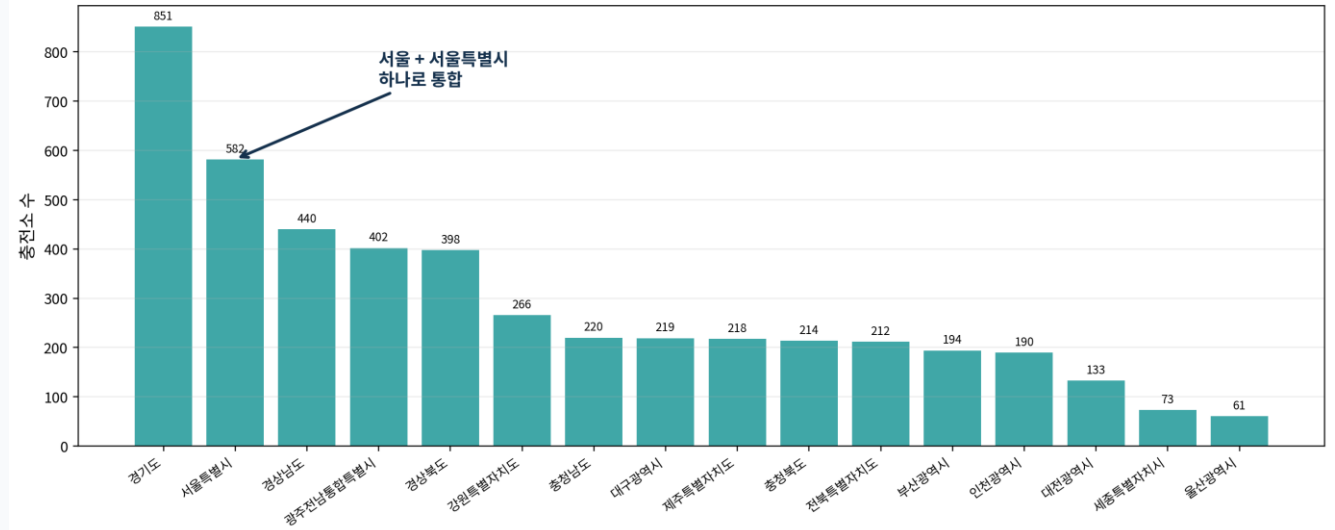
```
plt.figure(figsize=(12, 6))  
ax = 지역별_개수.plot(kind="bar")
```

```
plt.title("지역별 전기차 충전소 수",  
         fontsize=18, pad=15)  
plt.xlabel("지역", fontsize=12)  
plt.ylabel("충전소 수", fontsize=12)  
plt.xticks(rotation=35, ha="right")  
plt.grid(axis="y", alpha=0.25)
```

```
for i, value in enumerate(지역별_개수):  
    plt.text(i, value, str(value),  
            ha="center", va="bottom",  
            fontsize=10)
```

```
plt.tight_layout()  
plt.show()
```

전처리 후: 표준화된 지역별 충전소 수

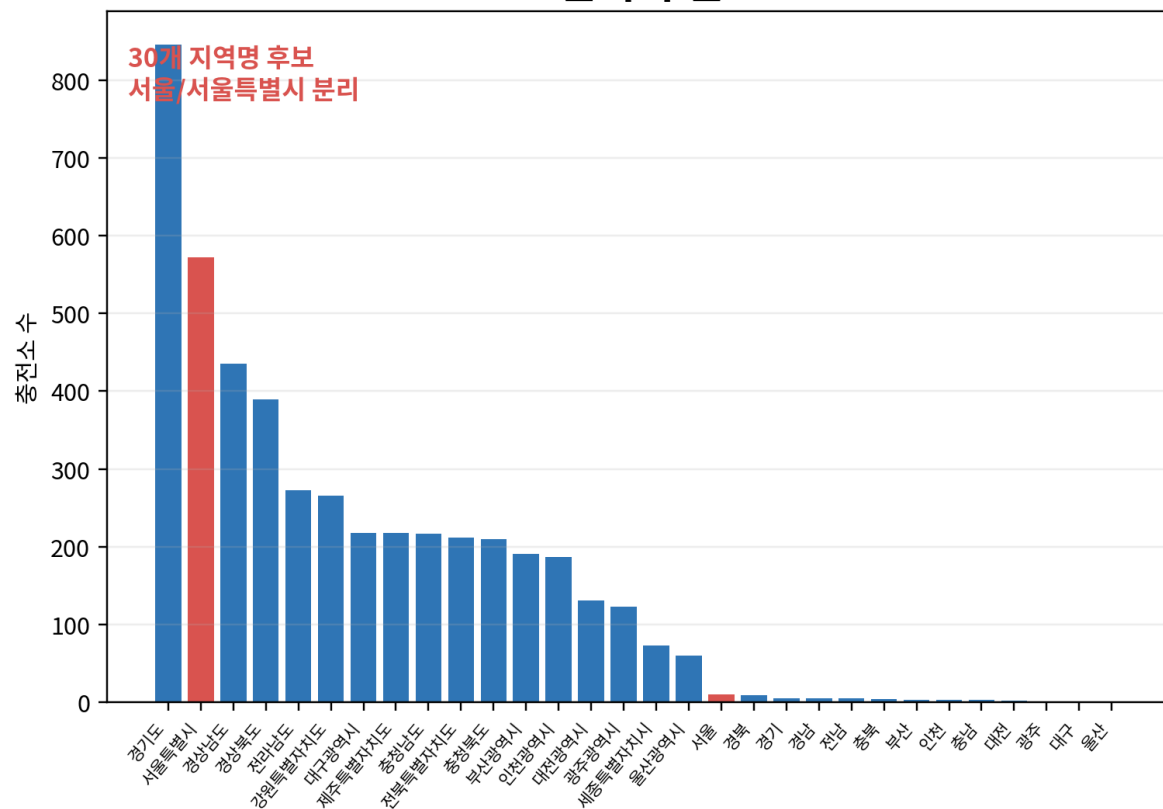


전처리 전과 후를 비교하자

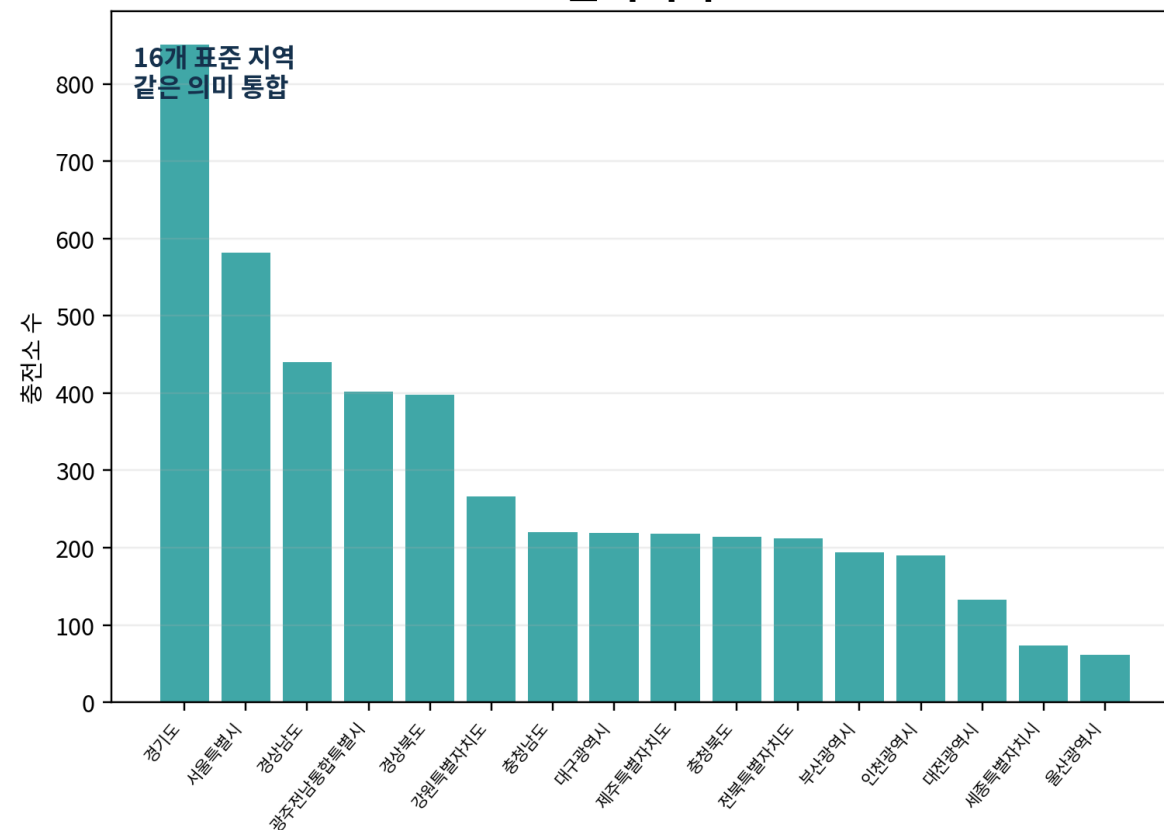
10

전처리의 필요성을 한 장에서 확인합니다.

전처리 전



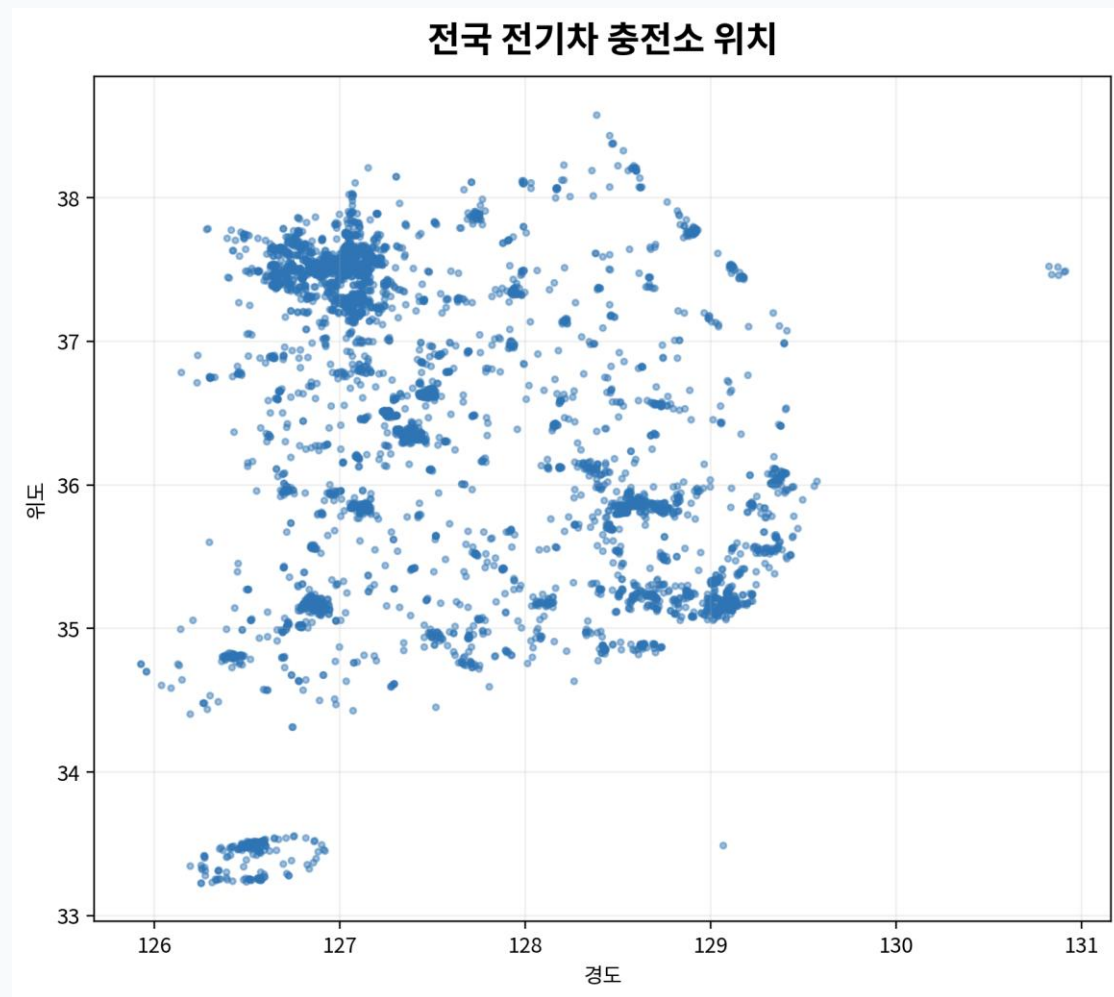
전처리 후



전처리 전: 같은 의미가 분리됨 → 전처리 후: 같은 의미가 통합되어 비교가 쉬워짐

STEP 7. 이제 위치를 시각화하자

막대그래프가 “몇 개?”를 보여 준다면, 지도 시각화는 “어디에?”를 보여 줍니다.



```
plt.figure(figsize=(10, 8))
```

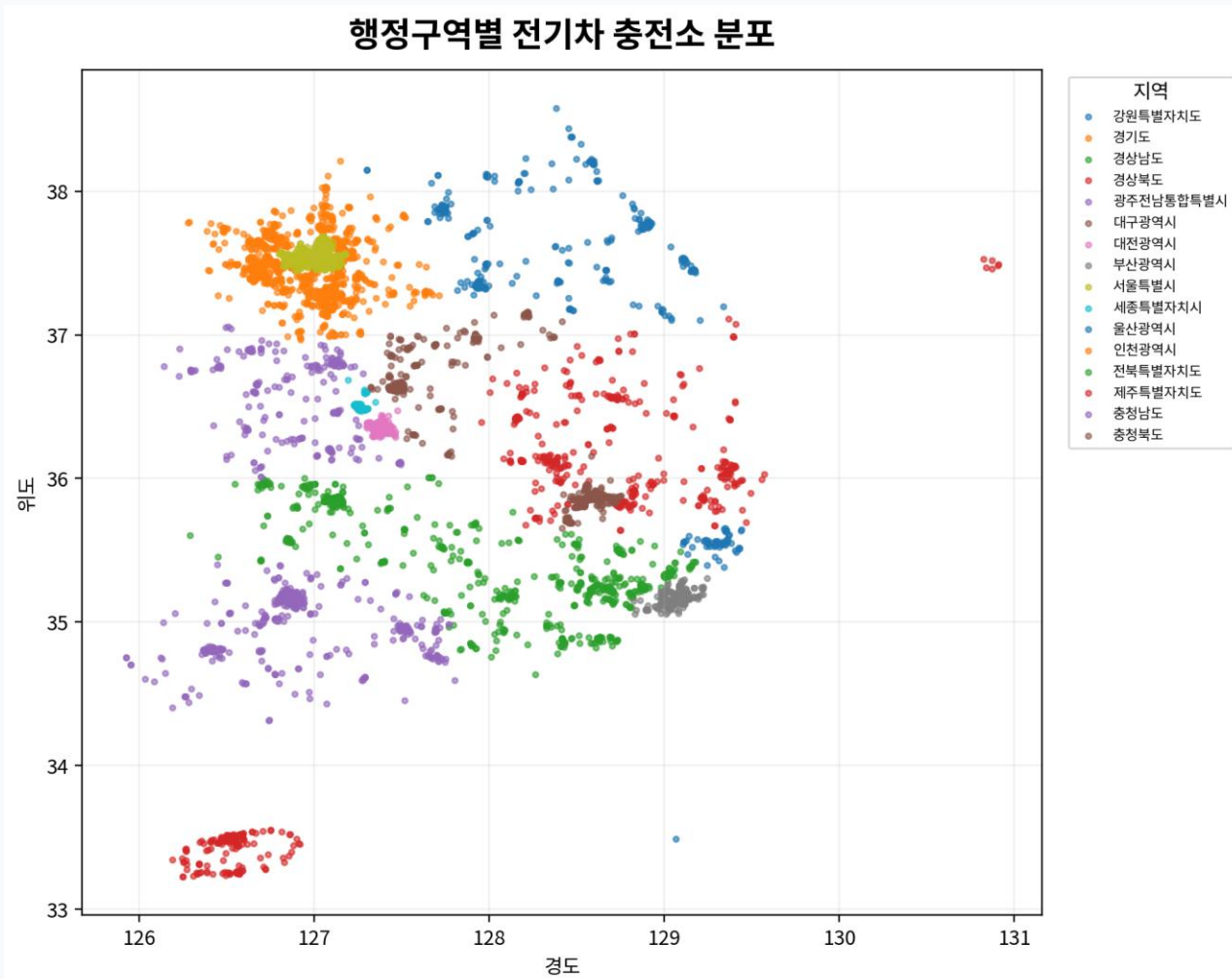
```
plt.scatter(  
    df["경도"],  
    df["위도"],  
    s=10,  
    alpha=0.5  
)
```

```
plt.title("전국 전기차 충전소 위치", fontsize=18)  
plt.xlabel("경도")  
plt.ylabel("위도")  
plt.grid(alpha=0.2)  
plt.tight_layout()  
plt.show()
```

경도는 x축, 위도는 y축으로 사용합니다.

STEP 8. 행정구역별로 색을 다르게 표시

groupby()로 지역별 데이터를 묶어 위치 분포를 비교합니다.



```
plt.figure(figsize=(12, 9))
```

```
for 지역, 데이터 in df.groupby("시도명"):
    plt.scatter(
        데이터["경도"],
        데이터["위도"],
        s=12,
        alpha=0.6,
        label=지역
    )
```

```
plt.title("행정구역별 전기차 충전소 분포",
          fontsize=20)
plt.xlabel("경도")
plt.ylabel("위도")
plt.grid(alpha=0.2)
plt.legend(title="지역", bbox_to_anchor=(1.02,
1),
           loc="upper left", fontsize=9)
plt.tight_layout()
plt.show()
```

색을 나누면
지역별 분포와 이상한 위치를 더 쉽게 찾을 수 있습니다.

STEP 9. Plotly로 대화형 지도처럼 탐색

마우스를 올리면 지역명과 좌표가 나타나 이상치 확인에 좋습니다.

```
import plotly.express as px

위치데이터 = df.dropna(subset=["위도", "경도"])

fig = px.scatter(
    위치데이터,
    x="경도", y="위도",
    color="시도명",
    hover_name="시도명",
    hover_data=["위도", "경도"],
    title="행정구역별 전기차 충전소 위치"
)

fig.update_traces(marker=dict(size=7, opacity=0.6))
fig.update_layout(width=900, height=900,
                  xaxis_title="경도", yaxis_title="위도")
fig.update_yaxes(scaleanchor="x", scaleratio=1)
fig.show()
```

hover_name: 마우스를 올리면 지역명 표시

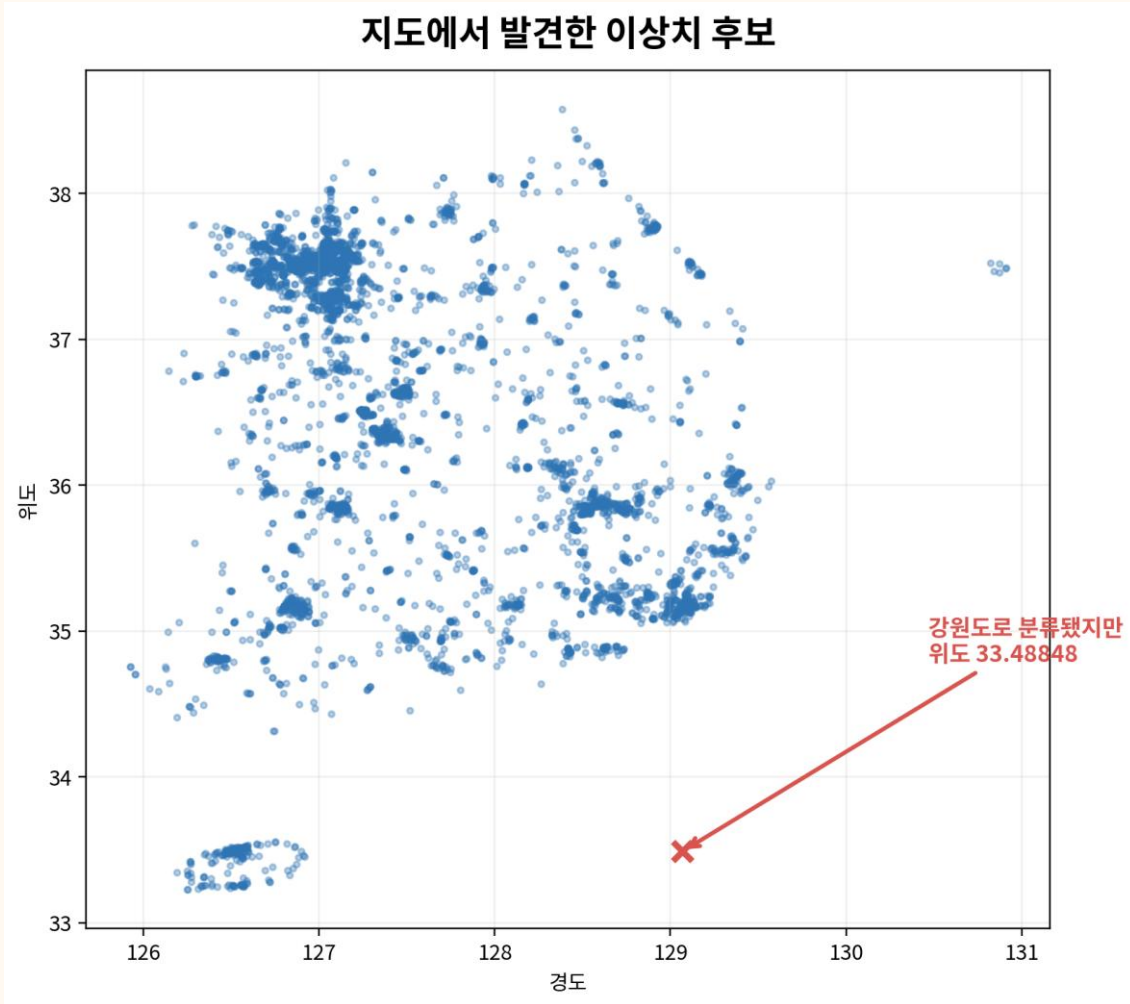
hover_data: 위도·경도 함께 확인

scaleanchor: 가로·세로 비율 유지

지도처럼 보면서 이상치 후보 탐색

STEP 10. 이상치를 발견하고 처리하기

강원특별자치도로 분류됐지만 위도 33.48848인 점을 확인합니다.



```
result = df[
    (df["시도명"] == "강원특별자치도") &
    (df["위도"].between(33.48, 33.50))
]
print(result)

# 실제 위치 확인 후 오류라고 판단되면 제거
df = df[
    ~(
        (df["시도명"] == "강원특별자치도") &
        (df["위도"] == 33.48848)
    )
]

# 제거 확인
print(len(df.loc[
    (df["시도명"] == "강원특별자치도") &
    (df["위도"] == 33.48848)
]))
```

이상치 발견 → 실제 위치 확인 → 오류 판단 → 제거 → 다시 시각화

수업 마무리

15

전처리는 분석을 방해하는 데이터를 없애는 것이 아니라, 분석 기준을 분명하게 만드는 과정입니다.

전처리 전
막대그래프

문제 발견
서울≠서울특별시

결측치·중복
확인

지역명
표준화

전처리 후
재시각화

위경도
지도 시각화

이상치
확인

오늘의 핵심 질문: 전처리 전 그래프만 보고 지역별 충전소 수를 판단하면 어떤 오류가 생길까?

학생 활동: 전처리 전·후 그래프를 보고 달라진 점 2가지를 설명하기

확장 활동: 지도에서 이상치 후보를 찾아 코드로 확인하기